

Reviewer Pack — Minimal Rank of Formal Language Invariants over AC^0 Circuit ...

Entry 8d353ffb284b · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-25 21:12:57 UTC

Minimal Rank of Formal Language Invariants over AC^0 Circuit Size

Entry ID: 8d353ffb284b **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-25 21:12:57 UTC

1. Conjecture

Field A (mathematical branch): Formal Language Theory (Deterministic Finite Automata) **Field B** (complexity object): Complexity Theory: AC^0 Circuit Complexity

Statement:

{‘sentence_1’: ‘For every deterministic finite automaton (DFA) with n states, let $R(DFA)$ be the rank of the regular language represented by the DFA.’, ‘sentence_2’: ‘Then, for all explicit functions in P , if the function can be computed by an AC^0 circuit of size $O(n^c)$ for some constant c , then $R(DFA) = \Theta(\log n)$.’, ‘sentence_3’: ‘This implies that a language represented by a DFA with rank $\Omega(n^2)$ cannot be computed by any AC^0 circuit.’}

Rationale (proposer’s reasoning):

{‘sentence_1’: ‘Formal language theory provides invariants for regular languages that can capture complexity properties not easily accessible in other branches of mathematics.’, ‘sentence_2’: ‘ AC^0 circuits are a fundamental model of computation with strong connections to complexity theory, especially for proving lower bounds.’, ‘sentence_3’: ‘The conjecture bridges these two fields by suggesting a quantitative relationship between DFA rank and AC^0 circuit size.’}

Taxonomy category: ACC_SIPSER (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 68a1668caaaf4113

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For a conjectured explicit function in P , if an AC^0 circuit of size $O(n^c)$ exists, then the rank $R(DFA)$ of its corresponding DFA is $\Theta(\log n)$. If any DFA with rank $\Omega(n^2)$ corresponds to such a function, the conjecture is falsified.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	0.95	UNCERTAIN	HITS
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "formal language theory" AND "AC⁰ circuit complexity" AND minimal rank"
- "deterministic finite automata" AND regular language AND AC⁰ circuit size" - "invariant rank" IN formal language theory AND AC⁰ circuit complexity

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=0, elapsed=0.0s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_dfa(n):
        states = list(range(n))
        transitions = {i: {} for i in states}
        start_state = 0
        accept_states = [n-1]

        for state in states:
            for symbol in range(2): # Assuming binary alphabet
                next_state = random.choice(states)
                transitions[state][symbol] = next_state

        return {
            'states': states,
            'transitions': transitions,
            'start_state': start_state,
            'accept_states': accept_states
        }

    def dfa_rank(dfa):
        n = len(dfa['states'])
        M = [[0] * n for _ in range(n)]

        for state, trans in dfa['transitions'].items():
```

```

        for symbol, next_state in trans.items():
            M[state][next_state] += 1

    rank = 0
    for i in range(n):
        if sum(M[i]) > 0:
            rank += 1

    return rank

def ac0_circuit_size(dfa):
    n = len(dfa['states'])
    # Simplified heuristic for AC0 circuit size
    return n * math.log2(n)

n_values = [5, 10, 15, 20, 30, 40]
results = []

for n in n_values:
    dfa = generate_dfa(n)
    rank = dfa_rank(dfa)
    circuit_size = ac0_circuit_size(dfa)

    if rank >= n**2:
        counterexample = f"DFA with n={n} has rank {rank}"
        return {
            "metric_name": "DFA Rank",
            "metric_value": rank,
            "instances_tested": len(n_values),
            "conjecture_holds": False,
            "counterexample": counterexample
        }

    results.append({
        "n": n,
        "rank": rank,
        "circuit_size": circuit_size
    })

mean_rank = sum(result['rank'] for result in results) / len(results)
std_rank = math.sqrt(sum((result['rank'] - mean_rank)**2 for result in results) / len(results))

return {
    "metric_name": "DFA Rank",
    "metric_value": mean_rank,
    "instances_tested": len(n_values),
    "conjecture_holds": True,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(arg) for arg in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []

```


The test only includes 6 instances, which is too small to be conclusive. The critic challenged the reliability of the results due to potential issues with scaling and representation. | next: Increase the number of instances tested and ensure they are representative of a broader range of cases. Investigate the scaling behavior of the conjecture with respect to n.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11907
2	preregistration	ollama_remote	glm4:latest	0	0	6129
3	novelty	ollama_remote	glm4:latest	0	0	4818
4	novelty	ollama_remote	glm4:latest	0	0	5281
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17313
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9477
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11774
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10309
9	critic	ollama_remote	glm4:latest	0	0	9288
10	judge	ollama_remote	glm4:latest	0	0	5709

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 92006 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in *research/audit/8d353ffb284b.jsonl*)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in *research/audit/8d353ffb284b.jsonl* for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: *research/pvsnp_sandbox/test_*.py* (per-cycle)

Replay tarball: *research/replay/8d353ffb284b.tar.gz* (if generated)